AQUASENTINEL: AGENTIC FLOOD RESPONSE AND EVACUATION OPTIMIZATION WITH DIGITAL TWIN

A Project Report

U23IT705 – Project

Submitted by

ABISHAK S (6178232311005)

BALAKARTHIKKEYAN B S (61782323110701)

GOWTHMAN J (61782323110702)

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

Information Technology

SONA COLLEGE OF TECHNOLOGY (AUTONOMOUS), SALEM– 636005

ANNA UNIVERSITY: CHENNAI 600 025

OCTOBER 2026

SONA COLLEGE OF TECHNOLOGY (AUTONOMOUS)
ANNA UNIVERSITY: CHENNAI – 600 025

BONAFIDE CERTIFICATE

Certified that this project titled **"AQUASENTINEL: AGENTIC FLOOD RESPONSE AND EVACUATION OPTIMIZATION WITH DIGITAL TWIN"** is the bonafide work of **"ABISHAK S (6178232311005), BALAKARTHIKKEYAN B S (61782323110701), GOWTHMAN J (61782323110702)"** who carried out the project work U23IT705 – Project under my supervision.

Signature of the Supervisor

NANDAKUMAR T
Assistant Professor
Information Technology
Sona College of Technology,
Salem – 636005

Head of the Department

AKLADESEVERY
Professor & Head
Information Technology
Sona College of Technology,
Salem – 636005

Submitted for the Project Viva-Voce Examination held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

Sona College of Technology

Department of Information Technology

U23IT705/U23ADS705 - PROJECT - DECLARATION FORM

Declaration by the Student(s)

We hereby declare that the project titled **"AQUASENTINEL: AGENTIC FLOOD RESPONSE AND EVACUATION OPTIMIZATION WITH DIGITAL TWIN"**
is an original work carried out by us under the guidance of **Nandakumar T**.
This project has not been submitted elsewhere for the award of any degree.

We also affirm that all the data and information used in this project are genuine and properly acknowledged. This project work is carried out with reference to the United Nations Sustainable Development Goals **SDG 11 – Sustainable Cities and Communities and SDG 13 – Climate Action.**

S.No	Register Number	Name	Signature
1.	6178232311005	ABISHAK S	
2.	61782323110701	BALAKARTHIKKEYAN B S	
3.	61782323110702	GOWTHMAN J	

Date: _____

Place: Sona College of Technology

ACKNOWLEDGMENT

We express our sincere gratitude to the Management and the Principal of Sona College of Technology, Salem, for providing the facilities and the encouraging academic environment that made this project possible.

We thank the Head of the Department of Information Technology for continuous support, timely reviews and valuable suggestions throughout the course of the project.

We are deeply grateful to our project supervisor for guidance on the problem formulation, the evaluation methodology and the honest reporting of results, and for patiently reviewing each stage of the work.

We also thank the project coordinators, the review panel members and all faculty members of the department for their feedback during the project reviews.

We acknowledge the providers of the open datasets and services used in this work: the INDOFLOODS flood-event database, the CAMELS-IND hydrometeorological dataset, the Open-Meteo weather and flood APIs and the OpenStreetMap community.

Finally, we thank our parents and friends for their constant encouragement and support.

ABSTRACT

Floods are the most frequent natural disaster in India, and the time between a rising river and an overwhelmed road network is often only a few hours. Existing tools usually forecast the hazard or display it on a map, but rarely connect the forecast to the decisions that follow: whether to evacuate, which shelter still has space, which road is open and which rescue team should go first.

This project presents AquaSentinel, an agentic real-time flood response and evacuation optimization system with a digital twin. A single Sense → Predict → Reason → Decide → Optimize → Respond pipeline collects live weather and river-stage data from Open-Meteo, scores it with machine-learning models trained on the INDOFLOODS and CAMELS-IND datasets, and combines the result with rainfall and stage-versus-danger terms in a documented risk engine. A gated policy chooses MONITOR, HUMAN REVIEW or AUTOMATED RESPONSE, and seven cooperating agents record every step as an auditable event. Flood-aware routes are computed with Dijkstra, A* and Yen's algorithm, zone-to-shelter assignment is solved as a QUBO problem with greedy, simulated-annealing and QAOA-inspired solvers, and SOS calls are clustered with K-means and served vulnerable-first. A digital twin and a five-dam monitor reuse the same pipeline on synthetic scenarios.

The system is built with React and Vite, FastAPI, MongoDB, scikit-learn, XGBoost, SHAP and NetworkX, with WebSocket updates and Docker deployment. The production 3-day flood forecast (XGBoost) reached a balanced accuracy of 0.898, recall of 0.902 and ROC-AUC of 0.965 on gauges never seen in training. In a seed-fixed benchmark of 30 synthetic scenarios, constraint-aware HQRL routing achieved 100 % safe evacuation with no capacity violations, against 90 % for shortest-path and RL-only routing.

Keywords: *flood forecasting, digital twin, multi-agent systems, evacuation routing, QUBO, explainable AI, XGBoost.*

TABLE OF CONTENTS

ACKNOWLEDGMENT **4**

ABSTRACT **5**

TABLE OF CONTENTS **6**

LIST OF FIGURES **9**

LIST OF TABLES **10**

LIST OF ABBREVIATIONS **11**

1. INTRODUCTION **12**

1.1 Background of the Project 12

1.2 Problem Statement 12

1.3 Motivation 13

1.4 Objectives 13

1.5 Scope of the Project 14

2. LITERATURE REVIEW **15**

2.1 Introduction 15

2.2 Review of Related Studies 15

2.2.1 Data-driven flood forecasting 15

2.2.2 Gradient boosting, random forests and explainability 16

2.2.3 Indian flood datasets 17

2.2.4 Digital twins for flood management 17

2.2.5 Evacuation routing and optimisation 17

2.2.6 Reinforcement learning for evacuation 18

2.2.7 LLM agents for disaster response 18

2.3 Techniques and Technologies Used in Existing Systems 19

2.4 Research Gap 19

2.5 Summary 20

3. CHAPTER 1 – NEED FOR PROPOSED TECHNIQUE **21**

3.1 Existing Techniques 21

3.2 Limitations of Existing Techniques 21

3.3 Proposed Technique 22

3.4 Objectives of the Proposed Technique 23

3.5 Advantages of the Proposed Technique 23

4. CHAPTER 2 – DESIGN OF PROPOSED SYSTEM AND ITS IMPLEMENTATION **24**

4.1 Introduction 24

4.2 System Requirements 24

4.2.1 Hardware Requirements 24

4.2.2 Software Requirements 24

4.3 System Architecture 25

4.4 System Components/Modules 27

4.4.1 Module 1 – Live Data Ingestion 27

4.4.2 Module 2 – Flood Prediction (Machine Learning) 27

4.4.3 Module 3 – Operational Risk and Decision Policy 27

4.4.4 Module 4 – Multi-Agent Coordination 28

4.4.5 Module 5 – Shelter, Route and Optimisation 28
4.4.6 Module 6 – SOS Triage and Rescue Operations 29
4.4.7 Module 7 – Digital Twin Simulation and Dam Monitor 29
4.4.8 Module 8 – HQRL Evacuation Benchmark 29
4.4.9 Module 9 – Voice SOS and LLM Agent Radio 30
4.4.10 Module 10 – Frontend Dashboard 30
4.5 Proposed Methodology 30
4.6 System Workflow 31
4.7 Algorithms/Techniques Used 32
4.7.1 Operational Risk Engine 32
4.7.2 Gated Decision Policy 33
4.7.3 Flood-Aware Routing 33
4.7.4 QUBO Zone-to-Shelter Assignment 33
4.7.5 SOS Clustering and Vulnerable-First Ordering 34
4.7.6 HQRL Constraint-Aware Routing 34
4.7.7 Explainability and Change Monitoring 34
4.8 Dataset/Data Collection 35
4.9 Data Preprocessing 35
4.10 Model/System Development 36
4.11 System Design Diagrams 37
4.12 Implementation 37
4.13 Coding/Program Implementation 38
4.14 User Interface / Application Screens 39
4.15 Chapter Summary 42
5. CHAPTER 3 – RESULTS AND DISCUSSION **43**
5.1 Introduction 43
5.2 Experimental Setup 43
5.3 Evaluation Metrics 43
5.4 Experimental Results 44
5.4.1 Version 1: INDOFLOODS Severe-Flood Classifiers 44
5.4.2 Versions 2 and 3: Peak Level and Severe Flag on Unseen Gauges 44
5.4.3 Version 4: 3-Day Flood Forecast on CAMELS-IND 45
5.4.4 Twenty-Model Benchmark 46
5.4.5 HQRL Routing Benchmark 47
5.4.6 Automated Tests 47
5.5 Comparison with Existing Techniques 48
5.6 Discussion of Results 48
5.7 Chapter Summary 49
6. CHAPTER 4 – CONCLUSION AND FUTURE SCOPE **50**
6.1 Summary of the Project 50
6.2 Conclusion 51
6.3 Future Scope 51
SDG GOALS **52**
PUBLICATIONS **53**
REFERENCES **54**

LIST OF FIGURES

Figure 4.1 Overall system architecture of AquaSentinel 26
Figure 4.2 Workflow of one pipeline cycle 32
Figure 4.3 Per-person rescue stage machine 34
Figure 4.4 HQRL constraint-aware evacuation routing pipeline 34
Figure 4.5 Deployment diagram (docker compose) 37
Figure 4.6 Live Command Center screen 40
Figure 4.7 Simulation Lab – Heavy Rainfall scenario results and Dam Monitor 41
Figure 4.8 Rescue Route – live map and SOS case roster 42
Figure 5.1 v1 classifier test metrics 44
Figure 5.2 v4 candidate comparison on validation data 45
Figure 5.3 v4 confusion matrix on unseen gauges 46
Figure 5.4 HQRL benchmark: safe evacuation and clearance time 47

LIST OF TABLES

Table 2.1 Summary of techniques in the reviewed literature 19
Table 4.1 Hardware requirements 24
Table 4.2 Software requirements 24
Table 4.3 Agents in the agent graph 28
Table 4.4 Datasets used for training and evaluation 35
Table 4.5 Model versions 36
Table 4.6 Repository file structure 38
Table 4.7 Application screens 40
Table 5.1 v1 test metrics on 910 held-out events 44
Table 5.2 v2/v3 results on 31 unseen gauges (788 events) 44
Table 5.3 v4 candidate models on validation (2009–2014, 349,645 rows) 45
Table 5.4 v4 test results on 49 unseen gauges (2015–2020, 52,994 rows) 45
Table 5.5 HQRL benchmark, 30 synthetic scenarios, seed 42 47
Table 5.6 Comparison with existing approaches 48

LIST OF ABBREVIATIONS

Abbreviation	Expansion
A*	A-star heuristic shortest-path search
AI	Artificial Intelligence
API	Application Programming Interface
AUC	Area Under the Curve
CAMELS-IND	Catchment Attributes and Meteorology for Large-sample Studies – India
CORS	Cross-Origin Resource Sharing
CWC	Central Water Commission
FRL	Full Reservoir Level
HQRL	Hybrid Quantum Reinforcement Learning
JWT	JSON Web Token
LLM	Large Language Model
ML	Machine Learning
MSL	Mean Sea Level
NDRF	National Disaster Response Force
PPO	Proximal Policy Optimization
PR-AUC	Area Under the Precision–Recall Curve
QAOA	Quantum Approximate Optimization Algorithm
QUBO	Quadratic Unconstrained Binary Optimization
RBAC	Role-Based Access Control
REST	Representational State Transfer
RF	Random Forest
ROC	Receiver Operating Characteristic
SA	Simulated Annealing
SDG	Sustainable Development Goal
SHAP	SHapley Additive exPlanations
SOS	Emergency distress request
UI	User Interface
WS	WebSocket
XAI	Explainable Artificial Intelligence
XGBoost	eXtreme Gradient Boosting

1. INTRODUCTION

1.1 Background of the Project

Floods affect more people in India than any other natural hazard. Monsoon rainfall, upstream reservoir releases and rapid urbanisation combine to produce events in which river stage, road accessibility and shelter demand all change within hours. The Sendai Framework for Disaster Risk Reduction [31] stresses that early warning is useful only when it leads to early action, and the 2030 Agenda [32] lists resilience to climate-related hazards as an explicit development goal.

Recent work has made data-driven flood forecasting practical. Global machine-learning models now issue reliable forecasts for extreme floods even in ungauged basins [22], and large Indian datasets such as INDOFLOODS [12] and CAMELS-IND [19] make it possible to train and evaluate models on Indian catchments. In parallel, digital twins — continuously updated virtual replicas of physical systems [6] — and large-language-model agents [36] are being explored as decision-support layers for emergency management.

AquaSentinel brings these strands together in one working prototype. It is a full-stack web platform for the Delhi Yamuna corridor and five major Indian dams that senses live conditions, predicts flood risk, reasons through a graph of cooperating agents, decides on an action under explicit safety gates, optimises shelters and routes, and coordinates the rescue response — and it runs exactly the same pipeline on synthetic scenarios as a digital twin.

1.2 Problem Statement

During a flood, emergency operators must answer several connected questions at once: How likely is flooding in the next few days? Is the risk high enough to evacuate automatically, or should a human review it? Which shelters are safe and still have space? Which roads are open, and what is the fastest safe route? Which of many simultaneous SOS calls should be served first, and by which team?

Existing tools typically answer one of these questions in isolation. Forecast portals show water levels but do not plan evacuations; routing tools ignore flood exposure; dashboards often show static or decorative numbers that are not tied to any live state. The problem addressed by this project is to build a single, auditable system in which every displayed value — risk, route, shelter, rescue progress — is computed from one authoritative state, and in which the path from sensor reading to rescue dispatch is transparent and testable.

1.3 Motivation

The motivation comes from three observations. First, recurring floods on the Yamuna in Delhi and in reservoir-fed basins such as the Krishna and Mahanadi show that the decision window is short, so operators benefit from automation that is safe by design rather than automation that acts silently. Second, machine-learning flood models are often reported with optimistic accuracy on random splits that leak information between gauges; we wanted a model whose performance is measured on gauges and years it has never seen. Third, emergency services increasingly expect natural-language interfaces; a voice SOS agent that asks only for the caller's name lowers the barrier for people in distress.

1.4 Objectives

1. To ingest live weather, rainfall and river-stage data, normalise it, and label every value with its real source or an explicit "data unavailable" state.
2. To train flood models with group- and time-aware validation, without target leakage, and to serve them with SHAP explanations of the model that actually produced each prediction.
3. To design a gated decision policy (MONITOR, HUMAN REVIEW, AUTOMATED RESPONSE) with configurable thresholds and an auditable operator review step.
4. To coordinate seven agents through a shared event graph that is persisted in MongoDB and streamed to the user interface over WebSocket.

5. To compute shelter assignments and evacuation routes from the current road state using Dijkstra, A*, Yen's k-shortest paths and QUBO-based optimisation.
6. To manage concurrent SOS requests with K-means clustering, vulnerable-first ordering and a per-person rescue stage machine driven by real route travel times.
7. To provide a digital twin and a dam monitor that reuse the same pipeline on synthetic scenarios, and to benchmark constraint-aware evacuation routing reproducibly.

1.5 Scope of the Project

The project covers the complete software system: data ingestion, model training and inference, risk and policy engines, agents, optimisation, rescue operations, simulation, a React dashboard, automated tests and Docker deployment. The operational geography is the Delhi NCR Yamuna corridor (Yamuna at ITO, Hathnikund and Okhla barrages, five public-facility shelters and rescue staging points) and five dams: Mettur, Srisailam, Nagarjuna Sagar, Hirakud and Bhakra.

The following are outside the scope. The system does not call authenticated India-WRIS or CWC gauge APIs and therefore never invents official gauge readings. Live traffic is represented by a curated road graph with runtime blockage flags rather than a commercial traffic feed. The QAOA solver is a quantum-inspired classical simulation, not execution on quantum hardware, and no quantum speed-up is claimed. The system is a research prototype and has not been deployed in a real emergency.

2. LITERATURE REVIEW

2.1 Introduction

This chapter reviews the literature that informs each layer of AquaSentinel: data-driven flood forecasting, explainable machine learning, Indian flood datasets, digital twins for flood management, evacuation routing and optimisation (classical, reinforcement-learning and quantum-inspired), and large-language-model agents for disaster response. For each area we note the techniques used and the limitations that motivate the design choices of this project.

2.2 Review of Related Studies

2.2.1 Data-driven flood forecasting

Mosavi, Ozturk and Chau [21] surveyed machine-learning flood prediction models and found that ensemble and hybrid methods consistently outperform single models, but also that many studies evaluate on short records and random splits, which makes their accuracy hard to transfer to new catchments. This observation directly shaped our validation design, in which test gauges are never seen during training.

Nearing et al. [22] showed in Nature that a machine-learning model trained on thousands of gauges can forecast extreme riverine floods in ungauged watersheds with reliability comparable to, or better than, a state-of-the-art physical system at lead times of several days. Their work demonstrates that large-sample training generalises across basins, which is the premise of our version-4 model trained on 242 Indian gauges.

Kirschstein and Sun [11] tested whether graph neural networks benefit from explicit river-network topology when forecasting discharge at many gauging stations, and found that the GNNs did not gain from the topology information and struggled with sudden discharge spikes. Wang et al. [33] argued that the tree-like structure of river networks causes over-squashing in GNNs, and showed that a reachability-based graph transformation extended the useful forecast horizon by about ten hours relative to an EA-LSTM baseline. Pan, Mohammadi and Taylor [24] combined a graph

convolutional recurrent network with residual error learning for 1–6 hour streamflow forecasting and reported gains over common baselines. Together these studies show that deep models are promising but that gauge-level features such as recent flow and rainfall remain the strongest predictors — the features our gradient-boosted model relies on.

Morales et al. [20] used machine learning to predict flood-related emergency (112) calls from meteorological and contextual variables. Their work connects the hazard forecast to the operational load on emergency services, the same link AquaSentinel makes between flood probability, SOS volume and rescue capacity.

2.2.2 Gradient boosting, random forests and explainability

Random forests [1] average many decorrelated decision trees and are robust to noisy tabular features. XGBoost [3] adds regularised second-order gradient boosting with efficient sparse handling, LightGBM [9] uses histogram-based leaf-wise growth for speed on large datasets, and CatBoost [26] uses ordered boosting to reduce prediction shift with categorical features. All four families are used in this project and compared under identical splits; the scikit-learn library [25] provides the common training and evaluation framework.

For explanation, Lundberg and Lee [16] unified several attribution methods under Shapley values (SHAP), and Lundberg et al. [17] introduced TreeSHAP, which computes exact Shapley values for tree ensembles in polynomial time. Wang et al. [34] examined the spatial pattern of SHAP values in a susceptibility model for hydro-morphological processes and showed that explanations can reveal when a model relies on spurious location signals. This is why AquaSentinel computes SHAP from the same booster that produced each displayed probability rather than from a surrogate model.

Liu, Ting and Zhou [14] proposed the Isolation Forest, which isolates anomalies with random partitions and needs no density model. We use it to flag abrupt jumps in rainfall, river level, dam level and flood probability on the command-centre cards.

2.2.3 Indian flood datasets

Kuntla and Saharia [12] released INDOFLOODS, an observational database of Indian flood events built by combining long-term station discharge with official warning and danger levels. Each event carries its start and end, peak level and discharge, volume, duration, time to peak and recession time, and is enriched with geomorphological, climatological, land-cover, soil and anthropogenic catchment attributes. AquaSentinel's version-1 to version-3 models are trained on INDOFLOODS.

Mangukiya et al. [19] published CAMELS-IND, which provides daily hydrometeorological time series and catchment attributes for catchments in Peninsular India. Because it contains daily observed streamflow alongside rainfall, temperature, evapotranspiration and soil moisture, it supports a true forecasting task — predicting whether flow will exceed a flood threshold in the next three days — which is the task of our version-4 model.

2.2.4 Digital twins for flood management

Grieves and Vickers [6] defined the digital twin as a virtual representation that mirrors a physical system through its life cycle and can be used to anticipate undesirable emergent behaviour. Underhill, McEwen and Coulthard [30] reported on the co-design of an environmental digital twin with flood-risk management professionals and found that practitioners value transparency, trust in data provenance and the ability to explore "what-if" scenarios more than visual realism. These findings support two design rules of this project: every value shown must trace to the current simulation state, and simulated data must always be labelled as simulation.

2.2.5 Evacuation routing and optimisation

Dijkstra's algorithm [4] computes single-source shortest paths on graphs with non-negative weights, A* [8] adds an admissible heuristic to focus the search, and Yen's algorithm [37] enumerates the k shortest loopless paths, giving operators alternatives when the best road closes. The NetworkX library [7] provides tested implementations of these algorithms for the road graph used in this project.

Kirkpatrick, Gelatt and Vecchi [10] introduced simulated annealing for combinatorial optimisation, and Lucas [15] showed how many NP-hard problems, including assignment and routing problems, can be written as Ising or QUBO models. Farhi, Goldstone and Gutmann [5] proposed the Quantum Approximate Optimization Algorithm (QAOA), a variational method that alternates cost and mixer operators. Cadeliña et al. [2] applied D-Wave quantum annealing to a QUBO formulation of vehicle routing for disaster-response resource allocation in Marikina City and reported improved distribution of resources. We follow the same QUBO modelling approach for zone-to-shelter assignment but run a classical simulation of QAOA and compare it honestly with greedy and simulated-annealing solvers.

2.2.6 Reinforcement learning for evacuation

Schulman et al. [28] introduced Proximal Policy Optimization (PPO), a stable policy-gradient method widely used in control problems. Liu et al. [13] coupled storm-surge models (ADCIRC and SWAN) with a deep Q-network to plan evacuation of a petrochemical industrial zone, and Raei, Safarzadeh and Wang [27] proposed congestion-aware multi-agent reinforcement learning for wildfire evacuation in which each intersection is an independent agent that balances efficiency against hazard avoidance. A common concern in these studies is that a learned policy may still select an unsafe route; our HQRL module therefore places a hard QUBO constraint layer and a deterministic validation checklist after the RL candidate generator.

2.2.7 LLM agents for disaster response

Xu et al. [36] reviewed 70 studies on large language models in disaster management and proposed a framework built on multi-modal data fusion, multi-source validation and multi-agent collaboration. Xie et al. [35] built a retrieval-augmented multi-agent LLM system for natural-hazard resilience and showed that grounding in domain data greatly improves the usefulness of answers. Sultimov et al. [29] demonstrated RESPOND, which couples AI flood forecasting with an agent-based population model whose agents use LLM reasoning. In AquaSentinel, LLMs are used only for narration (agent radio) and for the voice SOS front end; the decisions themselves come from

deterministic, testable code.

2.3 Techniques and Technologies Used in Existing Systems

Table 2.1 Summary of techniques in the reviewed literature

Area	Representative techniques	Studies
Flood forecasting	Large-sample ML, LSTM, GNN, graph-conv RNN	[22], [11], [33], [24], [21]
Tabular ML	Random forest, XGBoost, LightGBM, CatBoost	[1], [3], [9], [26]
Explainability	SHAP, TreeSHAP, spatial SHAP analysis	[16], [17], [34]
Anomaly detection	Isolation Forest	[14]
Datasets	INDOFLOODS, CAMELS-IND	[12], [19]
Digital twin	Virtual replica, co-designed environmental twin	[6], [30]
Routing	Dijkstra, A*, Yen k-shortest paths	[4], [8], [37], [7]
Optimisation	Simulated annealing, QUBO, QAOA, quantum annealing	[10], [15], [5], [2]
RL evacuation	PPO, DQN, multi-agent RL	[28], [13], [27]
LLM agents	RAG multi-agent, LLM-driven agent simulation	[36], [35], [29]
Emergency demand	ML prediction of emergency calls	[20]

2.4 Research Gap

The review reveals the following gaps that this project addresses:

- Forecasting studies rarely connect the probability to an operational decision, a shelter plan and a rescue dispatch in one system.
- Many reported flood-model accuracies use random row splits; few studies evaluate on unseen gauges and later years, which is how a deployed model is actually used.
- Explanations are frequently produced by a surrogate model rather than the model that made the displayed prediction.
- RL evacuation planners optimise speed but do not always enforce hard constraints such as closed roads and shelter capacity.
- Quantum-optimisation studies often claim advantage without a like-for-like

classical baseline on the same instances.

- Dashboards and digital twins often show static or decorative values; practitioners ask for provenance and clearly labelled simulation [30].

2.5 Summary

Machine-learning flood forecasting, explainable boosting models, graph search, QUBO optimisation, reinforcement learning and LLM agents are each mature enough to use, but they are rarely combined into a single auditable pipeline with honest evaluation. AquaSentinel is designed to close this gap: one shared pipeline, group- and time-aware model validation, explanations from the serving model, hard safety constraints after any learned component, and a digital twin that never presents simulated data as live.

3. CHAPTER 1 – NEED FOR PROPOSED TECHNIQUE

3.1 Existing Techniques

Threshold-based warning from gauge levels. Agencies compare the observed water level at a station with fixed warning and danger levels. The method is simple and trusted, but it is reactive: an alert is raised only when the river is already high, and it gives no guidance on shelters or routes.

Physical hydrological and hydraulic models. Rainfall–runoff and river-routing models simulate flow and inundation from physics. They are accurate when calibrated, but they need detailed inputs, are computationally heavy and are usually run by specialists rather than embedded in an operations dashboard.

Stand-alone machine-learning forecasts. Models such as random forests, gradient boosting and LSTMs predict flood occurrence or level from rainfall and catchment features [21]. They are fast, but many are validated with random splits, provide no explanation, and stop at a probability.

GIS dashboards and static evacuation plans. Pre-computed evacuation maps and dashboards show shelters and roads, but routes are not recomputed when a road floods or a shelter fills, and the displayed numbers are often not tied to live state.

Learned routing policies. Reinforcement-learning planners [13], [27] adapt to changing conditions but may choose unsafe routes when the reward does not encode every constraint.

3.2 Limitations of Existing Techniques

- Fragmentation: forecasting, decision, shelter planning, routing and rescue are separate tools, so operators reconcile them manually.
- Optimistic evaluation: random splits leak gauge identity and give accuracy that does not hold on new rivers or years.
- Opaque predictions: probabilities are shown without a faithful explanation of what drove them.

- Unsafe or silent automation: there is no explicit gate that stops automatic action when data, the model or the database is unhealthy.
- Static plans: routes and shelter choices do not react to blockages or capacity changes.
- No safe rehearsal: there is rarely a digital twin that runs the same logic on scenarios, so plans cannot be tested before a real event.

3.3 Proposed Technique

We propose an agentic, state-driven pipeline in which a single function, run_pipeline(mode, observations), executes Sense → Predict → Reason → Decide → Optimize → Respond for both live and simulation modes; only the data provider changes. The key elements are:

- A hybrid operational risk engine that keeps the trained model as a prior and adds documented rainfall and stage-versus-danger terms, so risk rises monotonically with the hazard.
- A gated decision policy with configurable monitor (0.50) and auto (0.60) thresholds and data-quality, model and database health gates.
- A seven-agent event graph whose events are stored in MongoDB and streamed over WebSocket.
- Flood-aware routing (Dijkstra, A*, Yen) and QUBO-based zone-to-shelter assignment solved by greedy, simulated annealing and a QAOA-inspired solver.
- K-means SOS clustering with vulnerable-first ordering and a per-person rescue stage machine whose stage durations equal route travel times.
- A constraint-aware HQRL routing benchmark (PPO candidates → QUBO constraints → solver → validation → XAI).
- A version-4 XGBoost 3-day flood forecast trained on CAMELS-IND with unseen-gauge testing and TreeSHAP explanations.

3.4 Objectives of the Proposed Technique

The proposed technique addresses each limitation directly. Fragmentation is removed because one pipeline produces the probability, the decision, the shelter plan, the routes and the rescue state, and every screen reads from that state. Optimistic evaluation is avoided by GroupShuffleSplit and GroupKFold by gauge for INDOFLOODS models and by a gauge- and time-separated split for CAMELS-IND. Opaque predictions are replaced by TreeSHAP computed on the serving model. Silent automation is prevented by the gates in the decision engine: if any gate fails, an automated response is downgraded to human review and the reason is recorded. Static plans are replaced by routes recomputed from the current road state at dispatch and again at pickup. Finally, the digital twin allows every scenario to be rehearsed with the same code.

3.5 Advantages of the Proposed Technique

- End-to-end traceability: every displayed value traces to one authoritative state and every decision is audit-logged.
- Honest, reproducible evaluation: metrics are read from evaluation files produced by the training scripts, and benchmarks use fixed seeds.
- Safety by construction: hard constraints and health gates sit after every learned or heuristic component.
- Adaptivity: routes and assignments are recomputed when roads block or shelters fill.
- Explainability: SHAP contributions, policy rules and validation checklists explain why an action was chosen.
- Accessibility: a voice SOS agent asks only for the caller’s name, and the dashboard works in light and dark themes.
- Portability: the whole stack runs locally or on a VPS with docker compose.

4. CHAPTER 2 – DESIGN OF PROPOSED SYSTEM AND ITS IMPLEMENTATION

4.1 Introduction

This chapter describes the design and implementation of AquaSentinel: the hardware and software requirements, the layered architecture, the modules, the methodology and workflow, the algorithms, the datasets and preprocessing, model development, design diagrams, the code organisation and the user interface.

4.2 System Requirements

The system was developed and tested on a single laptop and is packaged with Docker so that it can run on any Linux VPS. Requirements are listed below.

4.2.1 Hardware Requirements

Table 4.1 Hardware requirements

Component	Development machine used	Minimum recommended
Processor	Intel Core i5-1135G7 @ 2.40 GHz (11th Gen)	4-core x86-64 CPU
Memory (RAM)	8 GB (7.7 GB usable)	8 GB (16 GB for v4 retraining)
Graphics	Intel Iris Xe (integrated)	Not required (CPU-only training)
Storage	SSD	10 GB free (datasets, models, Docker images)
Network	Broadband Internet	Internet access for weather / flood APIs
Client	Any modern browser	Chrome, Edge or Firefox; microphone for voice SOS

4.2.2 Software Requirements

Table 4.2 Software requirements

Layer	Software / library	Purpose
Operating system	Windows 11 (development), Linux VPS (deployment)	Host platform
Frontend	React 19.1, React DOM, React Router 7.8, Vite 7.1	Single-page application and build tool
Maps and charts	Leaflet 1.9, React-Leaflet 5.0, Recharts 3.1	Flood and rescue maps, time-series charts
HTTP client	Axios 1.11, native WebSocket	REST calls and live updates

Layer	Software / library	Purpose
Backend	Python 3.11+, FastAPI ≥0.115, Uvicorn, Pydantic 2	REST / WebSocket API and validation
Database	MongoDB 7, Motor ≥3.6, PyMongo ≥4.10	Asynchronous persistence of state and audit logs
Machine learning	scikit-learn ≥1.6, XGBoost ≥2.1, LightGBM ≥4.0, CatBoost ≥1.2, joblib	Training and inference
Data and XAI	pandas ≥2.2, NumPy ≥2.0, SHAP ≥0.46	Feature engineering and explanations
Graphs	NetworkX ≥3.0	Road graph, Dijkstra, A*, k-shortest paths
Security	python-jose (JWT), passlib/bcrypt, SlowAPI	Operator login, hashing, rate limiting
Reporting	fpdf2 ≥2.8	After-action PDF report
External services	Open-Meteo, OpenWeather (optional), Groq LLM, Google Gemini	Weather, flood discharge, agent radio, voice SOS
Testing	pytest, pytest-asyncio, Vitest 3.2, Testing Library, jsdom	Backend and frontend tests
Deployment	Docker, docker compose, nginx	Containers and reverse proxy

4.3 System Architecture

AquaSentinel follows a five-layer architecture, shown in Figure 4.1. The presentation layer is a React single-page application with four screens. It talks to the backend through REST endpoints under /api and a WebSocket hub at /ws that pushes typed events (weather_update, risk_update, agent_update, route_update, rescue_update, simulation_state and others) and heartbeats.

The application layer is a FastAPI service. Twelve routers (health, live, ml, agents, response, simulation, hqrl, emergency, sources, rescue_desk, voice_agent and dams) expose the functionality. At its centre is the shared pipeline run_pipeline(mode, observations): data normalisation, the feature engine, ML prediction, the operational risk engine, the decision policy and the agent graph, followed by the shelter engine, route engine, QUBO optimiser, SOS clustering and the rescue stage machine. A background task runs the live pipeline every 90 seconds. Live and simulation snapshots are stored separately so that synthetic ticks can never appear as live intelligence.

The data layer holds MongoDB collections (observations, agent_events, incidents, policy and review audit logs, rescue teams, shelters, benchmark results), the trained model artefacts and evaluation files, and the training datasets, which are never used as live telemetry. External sources are Open-Meteo for weather and river discharge, OpenWeather when a key is configured, OpenStreetMap tiles, and the Groq and Gemini LLM APIs. All API keys stay in the server-side .env file; the browser talks only to the backend.

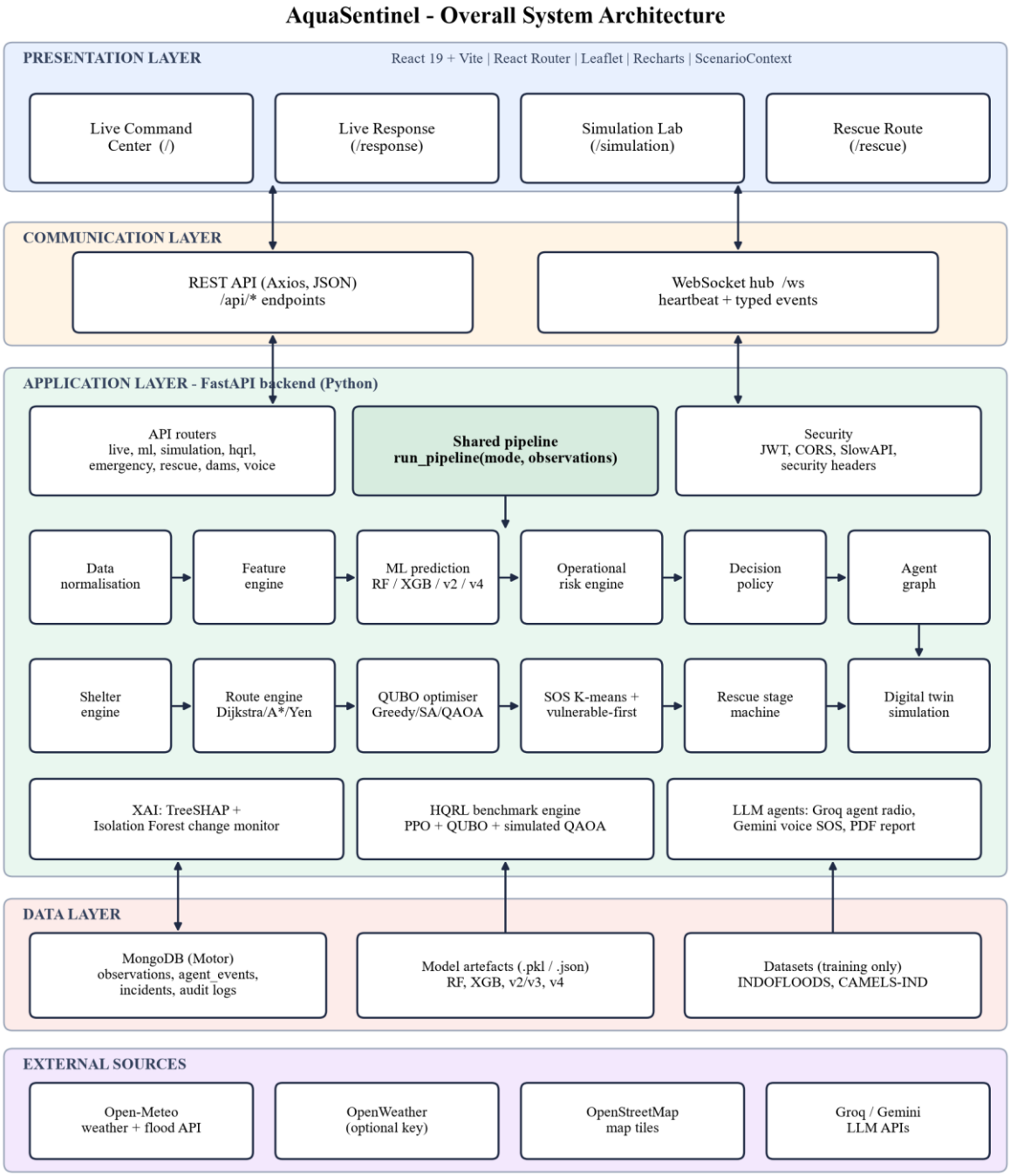


Figure 4.1 Overall system architecture of AquaSentinel

4.4 System Components/Modules

The system is organised into ten modules. Each module lives in its own package or file, so it can be tested in isolation.

4.4.1 Module 1 – Live Data Ingestion

The weather service (backend/services/weather_service.py) fetches current conditions, 24-hour rainfall and a 10-day precipitation forecast for Delhi. OpenWeather is used when OPENWEATHER_API_KEY is set; otherwise Open-Meteo is used without a key [23]. The hydrology service (hydro_service.py) calls the Open-Meteo Flood API for river discharge at Yamuna-ITO, Hathnikund Barrage and Okhla Barrage and maps it onto the configured danger level of 205.80 m. If every provider fails the service returns DATA SOURCE UNAVAILABLE instead of inventing values, and each observation is stored with its source label.

4.4.2 Module 2 – Flood Prediction (Machine Learning)

The inference module (backend/ml/inference.py) loads the model artefacts once at start-up. Version 1 provides a Random Forest and an XGBoost classifier for severe floods on INDOFLOODS; a dual-model view compares both and flags disagreement above the shared gap threshold of 0.15. Version 2/3 is a two-stage model: a LightGBM peak-flood-level regressor (tail-weighted in v3) followed by a CatBoost severe-flood classifier. Version 4 (forecast_v4.py) is the XGBoost 3-day flood forecast trained on CAMELS-IND; because no live observed-gauge feed is available, it is served as a historical replay on real gauge-days, returning the probability, what actually happened in the next three days and the TreeSHAP contributions from the same booster.

4.4.3 Module 3 – Operational Risk and Decision Policy

The operational risk engine (risk_engine.py) produces the probability used by the policy. The decision engine (decision_engine.py) maps that probability to an action using the thresholds in backend/thresholds.py, which is the single source of truth served at GET /api/config/thresholds. The policy is described in Section 4.7.

4.4.4 Module 4 – Multi-Agent Coordination

The agent graph (backend/agents/graph.py) holds seven agents, each with memory (status, last event, current action, timestamp and state). Every emit() writes an agent_events document in MongoDB and broadcasts an agent_update over WebSocket. Table 4.3 lists the agents.

Table 4.3 Agents in the agent graph

Agent	Input	Output event
Weather	Live or simulated weather	WEATHER_UPDATE
Dam	River / dam observations	DAM_UPDATE
Flood Risk	ML and operational probability	FLOOD_RISK_UPDATE, DECISION_REQUIRED
Traffic	Road flood exposure vs probability	Blocked-road set
Shelter	Capacity, risk and distance	SHELTER_SEARCH
Evacuation	Policy, routes and QUBO assignment	ROUTE_OPTIMIZATION, EVACUATION_PLAN
Rescue	SOS and automatic response	RESCUE_RESPONSE, SOS

An agent-radio layer (agents/dialogue.py) turns threshold crossings into short spoken-style messages between the Rescue, Ambulance, Disaster Team and Administrator agents using Groq-hosted LLMs. The LLM only narrates; it never changes a decision.

4.4.5 Module 5 – Shelter, Route and Optimisation

The shelter engine filters out shelters that are unsafe, closed, full, inaccessible or have flood risk of 0.65 or more (or 0.45 or more when the zone risk is at least 0.7), then scores the rest by distance, free seats, traffic and risk. The route engine builds a weighted graph of Delhi road nodes, removes blocked edges and computes flood-aware routes. The optimiser (backend/optimization/solvers.py) assigns population zones to shelters as a QUBO problem and solves it with four methods (greedy, Dijkstra proxy, simulated annealing and QAOA-inspired); the UI always shows the method actually used. An Algorithm Arena compares Dijkstra by distance only, Dijkstra flood-aware, A* flood-aware and greedy straight-line for every SOS.

4.4.6 Module 6 – SOS Triage and Rescue Operations

Concurrent SOS requests are clustered with K-means (sos_cluster.py) and clusters are matched to rescue teams by minimising people-weighted distance (an exhaustive permutation search when there are at most seven teams). Within the queue, priority.py orders people vulnerable-first: children (age < 12) and elderly people (age ≥ 60) first, then by water depth (chest/neck/shoulder before waist). The rescue stage machine (person_ops.py, Figure 4.3) moves each person through queued → assigned → en_route → to_shelter → rescued, with stage durations equal to real route travel times. The rescue desk manages ten ambulance units and five shelters with about 10,000 places in total.

4.4.7 Module 7 – Digital Twin Simulation and Dam Monitor

The simulation engine (backend/simulation/engine.py) evolves a scenario tick by tick: rainfall rises, river and dam stage rise, ML probability updates, roads with high flood exposure block, shelter occupancy grows and SOS count rises. Each tick calls run_pipeline(mode="simulation"), so the twin uses exactly the same logic as live mode, and every simulation screen is labelled SIMULATION MODE — SYNTHETIC DATA. The Dam Monitor (dam_risk.py) covers Mettur (Cauvery), Srisailam (Krishna), Nagarjuna Sagar (Krishna), Hirakud (Mahanadi) and Bhakra (Sutlej). It computes live-storage fill as (level − dead storage) / (FRL − dead storage), maps fill to bands (MODERATE ≥ 50 %, HIGH ≥ 70 %, VERY HIGH ≥ 85 %, CRITICAL ≥ 95 %) and responses (MONITOR, ALERT, PREPARE EVACUATION, EVACUATE), and runs a bounded water-balance simulation. The scenario catalogue offers dam scenarios (normal operation, heavy rainfall, extreme rainfall, emergency release, worst-case inflow, drought/low inflow), simulation scenarios (road blockage, shelter congestion, multiple SOS surge) and two HQRL benchmark scenarios.

4.4.8 Module 8 – HQRL Evacuation Benchmark

The HQRL engine (backend/simulation/hqrl_demo.py) is a synthetic research prototype for constraint-aware evacuation routing on a 15-node graph with three

population zones and three shelters. A PPO-style policy proposes the top-K route candidates; a QUBO encodes hard constraints (road closures, shelter capacity, vehicle availability and accessibility); a simulated QAOA solver, with a classical QUBO fallback, selects the plan; a deterministic validation checklist confirms it; and an XAI panel explains each accept or reject decision. A human-in-the-loop gate holds automatic dispatch when data sources conflict. The engine benchmarks six methods over fixed-seed scenarios and runs an ablation without the safety layer (Figure 4.4).

4.4.9 Module 9 – Voice SOS and LLM Agent Radio

The voice agent (services/voice_agent.py) uses Google Gemini with a single tool, submit_flood_sos. The agent asks only for the caller's full name; location comes from the device and other fields use defaults. When the name is known the tool registers the SOS, assigns the nearest available team and a shelter, and the agent reads back the assignment in English.

4.4.10 Module 10 – Frontend Dashboard

The React application has four routes. The Live Command Center (/) shows live weather, hydrology, ML and operational flood probability, agents, the map and measured latencies. Live Response (/response) shows incidents, the policy decision with approve/reject controls, shelters, routes and optimisation. The Simulation Lab (/simulation) holds the scenario picker, inputs, the single RUN SCENARIO flow, the Dam Monitor, the ML benchmark and the HQRL benchmark; its state is owned by ScenarioContext. Rescue Route (/rescue) shows the live rescue map with animated units, the live SOS roster and the event feed. Leaflet icons are cached and units are animated with one shared requestAnimationFrame loop so the map stays smooth while the page re-renders every 1.2 s.

4.5 Proposed Methodology

1. Data acquisition: collect live weather, rainfall and river discharge from Open-Meteo (and OpenWeather if configured); load INDOFLOODS and CAMELS-IND for training only.

2. Preprocessing and features: clean the data, remove leakage columns, build rainfall-accumulation, flow-ratio, soil-moisture, seasonal and catchment features.
3. Model training and selection: train candidate models with group- and time-aware splits, choose on validation, and evaluate once on unseen gauges.
4. Risk estimation: combine the model prior with rainfall and stage-versus-danger terms in the operational risk engine.
5. Decision: apply the gated policy to choose MONITOR, HUMAN REVIEW or AUTOMATED RESPONSE; log operator reviews.
6. Coordination: let the agent graph emit events and stream them to the UI.
7. Optimisation: filter and score shelters, compute flood-aware routes and solve the QUBO assignment.
8. Response: cluster SOS calls, order them vulnerable-first and run the rescue stage machine.
9. Rehearsal and evaluation: run the same pipeline on digital-twin scenarios and benchmark algorithms with fixed seeds.

4.6 System Workflow

Figure 4.2 shows the workflow of one pipeline cycle. A cycle starts from the live ticker or a simulation tick, senses the environment, builds features, predicts risk, checks the health gates, lets the agents reason, decides on an action, optimises shelters and routes, responds by dispatching teams and pushing updates to the UI, and finally stores the snapshot and audit log.

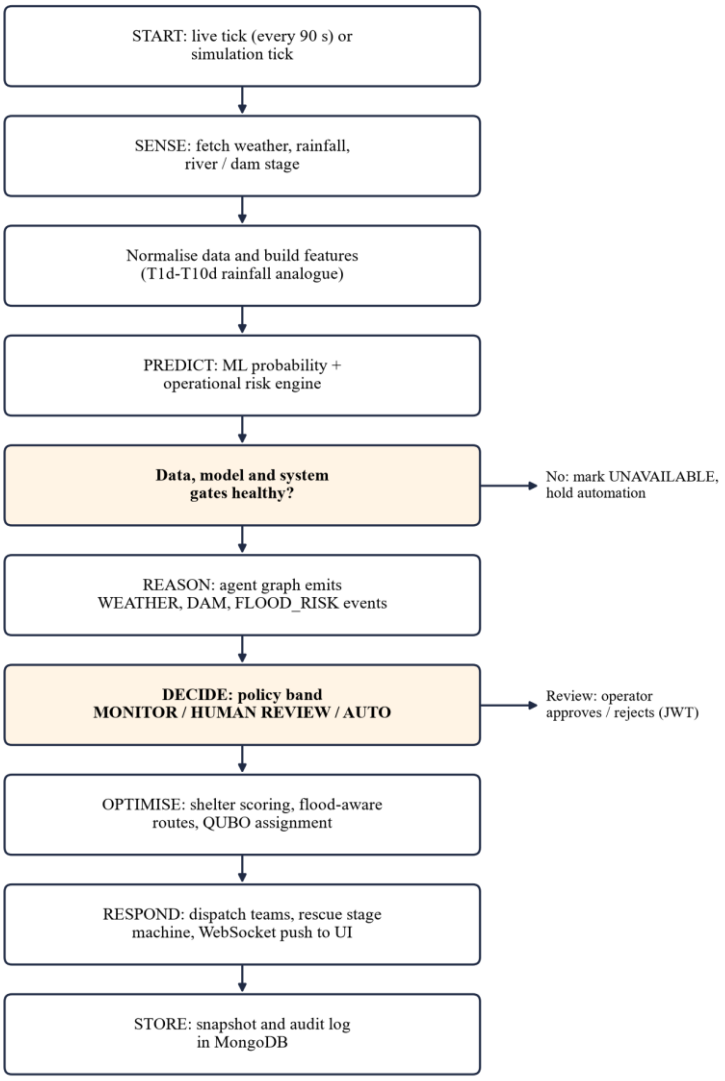


Figure 4.2 Workflow of one pipeline cycle

4.7 Algorithms/Techniques Used

4.7.1 Operational Risk Engine

The INDOFLOODS severe-versus-flood label is largely a station property, so the trained classifier barely changes with rainfall. The operational risk engine therefore keeps the ML score as a prior and adds hazard terms that must increase as the hazard increases:

$$p = clip(\ 0.16 + 0.50 \cdot \sigma_rain + 0.20 \cdot h_stage + 0.14 \cdot p_ML\ ,\ 0\ ,\ 1\ )$$

$$\sigma_rain = 1 / (1 + exp(-(R - 38) / 12)), \quad h_stage = mean(\ clip((pct - 70) / 30,\ 0,\ 1)\ )$$

where R is the 24-hour rainfall in mm and pct is the river or dam stage as a percentage of its danger level. The coefficients are documented project settings, not a claim of

extra ML accuracy, and the engine is tested to be monotonic in rainfall.

4.7.2 Gated Decision Policy

If the probability is unavailable, the action is UNAVAILABLE. If an operator override is active, the action is HUMAN_OVERRIDE. Otherwise $P < 0.50$ gives MONITOR, $0.50 \leq P < 0.60$ gives HUMAN_REVIEW, and $P \geq 0.60$ gives AUTOMATED_RESPONSE — but only if automation is enabled and the data-quality, model and database gates all pass; otherwise it is downgraded to HUMAN_REVIEW with the reason "safety gate blocked automation". Severity labels use 0.35 (moderate), 0.50 (high) and 0.70 (critical).

4.7.3 Flood-Aware Routing

Each road edge (u, v) receives a cost that inflates its length by traffic and flood exposure; blocked or inaccessible edges are removed:

$$cost(u, v) = d_{uv} \cdot (1 + 0.8 \cdot traffic + 1.4 \cdot flood_exposure)$$

Travel time is cost / 25 km/h. Dijkstra's algorithm [4] returns the cheapest path, A* [8] uses the haversine distance to the destination as heuristic, and Yen's algorithm [37] provides alternative routes.

4.7.4 QUBO Zone-to-Shelter Assignment

Let $x_{ij} = 1$ if zone i is assigned to shelter j. The cost of each pair is

$$c_{ij} = 0.30 \cdot travel + 0.25 \cdot distance + 0.20 \cdot (8 \cdot flood_risk_j) + 0.10 \cdot (5 \cdot traffic) + 0.15 \cdot overload_{ij}$$

and penalties enforce one shelter per zone, shelter capacity and the exclusion of unsafe shelters [15]. The greedy solver picks the cheapest feasible column per zone; simulated annealing [10] searches reassignments; the QAOA-inspired solver [5] runs variational cost and mixer phases over one-hot bitstrings on a classical simulator. Runtime, cost, distance, evacuation time, risk exposure, utilisation and constraint violations are recorded for each method.

4.7.5 SOS Clustering and Vulnerable-First Ordering

K-means [18] (n_init = 10, random_state = 42) groups SOS locations; clusters are matched to teams by minimum people-weighted distance. A vulnerability tier (child or elderly first) and a water-depth tier then order the dispatch queue. One SOS is dispatched every 3 wall-clock seconds, crews take 2 seconds to mobilise, and the demo clock runs 120 times faster than real time (1 s = 2 simulated minutes).

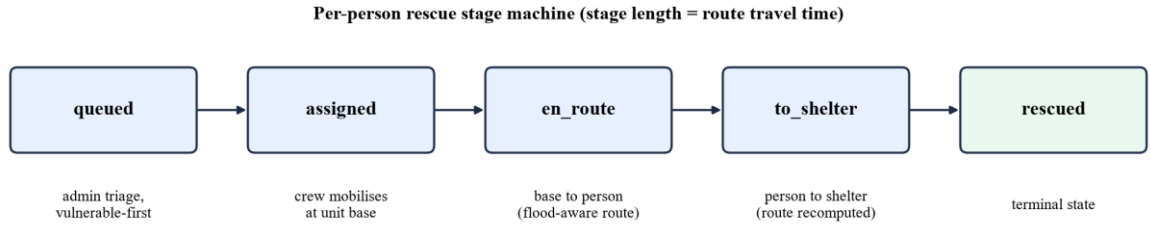


Figure 4.3 Per-person rescue stage machine

4.7.6 HQRL Constraint-Aware Routing

Figure 4.4 shows the HQRL pipeline. The policy [28] proposes candidate routes; the QUBO layer removes candidates that use closed roads, exceed shelter capacity, lack vehicles or violate accessibility; the solver selects among the remaining candidates; and the validation checklist (route present, road status, shelter capacity, vehicle availability, accessibility, priority constraints, graph version, data freshness) must pass before a recommendation is shown.

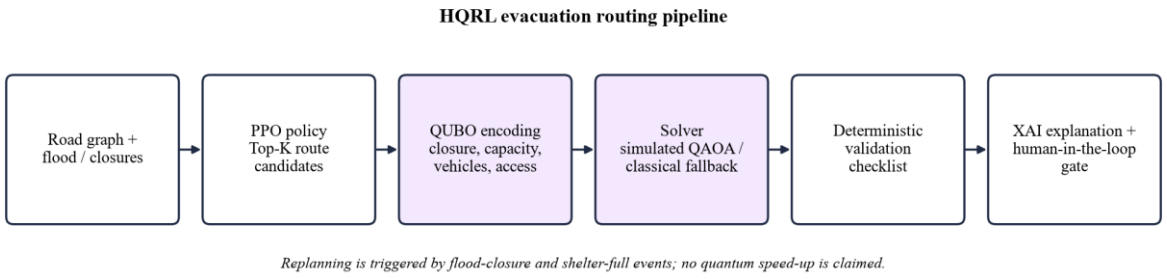


Figure 4.4 HQRL constraint-aware evacuation routing pipeline

4.7.7 Explainability and Change Monitoring

TreeSHAP [17] is computed on the Random Forest, on the v2 regressor and on the v4 XGBoost booster (pred_contribs in log-odds units), always on the model that produced

the displayed number. An Isolation Forest [14] (40 trees, contamination 0.12) together with a rate-of-change rule flags sudden jumps in rainfall, river level, dam level and flood probability.

4.8 Dataset/Data Collection

Table 4.4 Datasets used for training and evaluation

Dataset	Contents	Use in the project
INDOFLOODS [12]	4,548 flood events (2,919 Flood, 1,629 Severe Flood); event-scale precipitation T1d–T10d; catchment characteristics	v1 RF/XGBoost severe-flood classifiers; v2/v3 peak-level regressor and classifier
CAMELS-IND [19]	Daily rainfall, temperature, humidity, evapotranspiration, 4-layer soil moisture and observed streamflow with catchment attributes; 242 gauges used	v4 3-day flood forecast (production model)
Open-Meteo (live)	Current weather, 24-h rainfall, 10-day forecast, river discharge	Live inference only — never used for training
Curated geography	Delhi road graph, 5 shelters, rescue staging points, 5 dams	Routing, shelter assignment, Dam Monitor

Two other files in the dataset folder (state-level daily rainfall and an India weather workbook) were examined and excluded because they share no gauge, coordinate or state key with the INDOFLOODS catchment table.

4.9 Data Preprocessing

- Leakage removal: columns describing the outcome of an event — peak flood-level date, number of peaks, peak discharge and its date, flood volume, event duration, time to peak and recession time — are excluded.
- Group leakage removal: gauge identity, per-gauge priors and per-gauge z-scores were dropped after v1 so that the models generalise to unseen catchments.
- Rainfall features (INDOFLOODS): cumulative T1d–T10d, log transforms, month, day of year, monsoon flag, recent-intensity and build-up ratios.
- Forecast features (CAMELS-IND, 50 in total): lagged rainfall (0–3 days), rainfall sums over 3, 7, 14, 30 and 90 days, 7-day maximum, temperature, humidity, PET/AET, soil moisture in four layers and their changes, flow ratios to the gauge flood level (lags, changes, 7- and 30-day means, 7-day peak), log

flow, seasonal sine/cosine and static catchment attributes.

- Target (v4): 1 if observed streamflow reaches the gauge's training-period 95th percentile on any of the next three days.
- Splits: v1–v3 use GroupShuffleSplit 80/20 by gauge (3,760 / 788 events; 124 / 31 gauges) and GroupKFold(5) by gauge; v4 trains on seen gauges 1980–2008, validates on 2009–2014 and tests on 20 % unseen gauges for 2015–2020.
- Excluded from v4: CAMELS hydrological signatures computed over the full record (they include test years) and any future rainfall.

4.10 Model/System Development

Models were trained with the scripts in the scripts folder (train_flood_model.py through train_flood_model_v4_full.py, and benchmark_20_models.py). Each script writes the model (.pkl), its metadata and an evaluation JSON file; the backend reads these files and never recomputes or edits the metrics. Earlier versions are preserved, and new versions are added alongside them.

Table 4.5 Model versions

Version	Dataset and task	Selected model	Validation
v1	INDOFLOODS: Severe vs Flood	Random Forest (XGBoost compared)	Held-out 910 events
v2	INDOFLOODS: peak level + severe flag	LightGBM (log target) + CatBoost	GroupKFold(5), holdout of 31 gauges
v3	As v2, tail-weighted regressor	LightGBM tail-weighted + CatBoost	Same as v2
v4	CAMELS-IND: flood in next 3 days	XGBoost depth 6, refit on all training rows; threshold 0.45	Time split + unseen gauges

For v4, seven candidates (logistic regression, random forest, histogram gradient boosting, XGBoost with depth 4, 6 and 8, and depth-6 XGBoost refit on all 1,641,197 training rows) were compared on the 349,645 validation rows. The selection rule was maximum validation balanced accuracy with PR-AUC as tie-break, and the decision threshold (0.45) was also chosen on validation, so the test set was used only once.

4.11 System Design Diagrams

The architecture diagram (Figure 4.1), the workflow (Figure 4.2), the rescue stage machine (Figure 4.3) and the HQRL pipeline (Figure 4.4) were presented above. Figure 4.5 shows the deployment diagram.

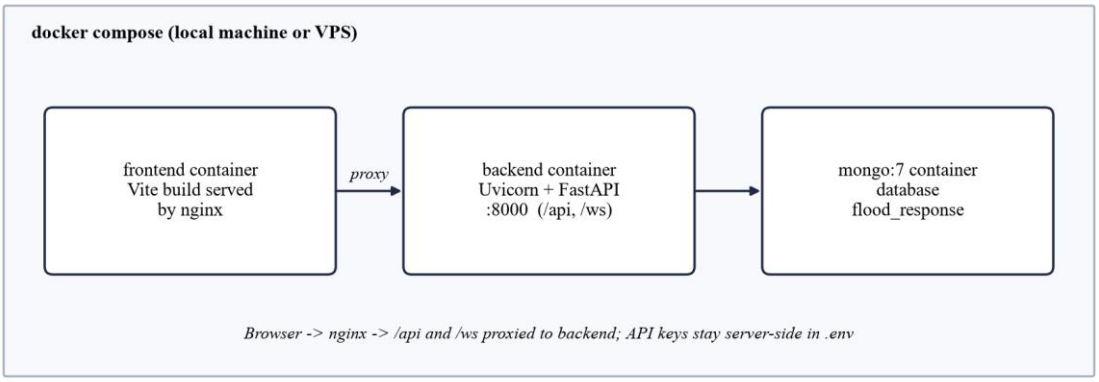


Figure 4.5 Deployment diagram (docker compose)

4.12 Implementation

The backend is an asynchronous FastAPI application. On start-up the lifespan handler connects to MongoDB, loads the models, seeds shelters and teams if the database is empty, and starts the 90-second live ticker. If MongoDB is unavailable the APIs still run and return explicit unavailable states. Requests are validated with Pydantic, CORS is restricted to configured origins, SlowAPI provides rate limiting, operators log in with JWT for review actions, and every response carries security headers (X-Content-Type-Options, X-Frame-Options, Referrer-Policy, Permissions-Policy and Content-Security-Policy).

The frontend is built with Vite. API calls go through services/api.js; a single shared WebSocket client (services/ws.js) reconnects with exponential back-off up to 15 seconds and sends heartbeats. The /simulation page keeps one authoritative state in ScenarioContext, and the live rescue view model in state/rescueOps.js derives unit positions from the stage start time, stage duration and route polyline without storing positions anywhere.

For deployment, docker-compose.yml starts three containers — the nginx-served

frontend, the Uvicorn backend and MongoDB 7 — and scripts/deploy-vps.sh installs Docker, clones the repository and starts the stack on a VPS.

4.13 Coding/Program Implementation

GitHub repository: https://github.com/balasan12626/MINIPROJECT

The repository is organised as follows:

Table 4.6 Repository file structure

Path	Contents
backend/main.py	FastAPI app, lifespan, routers, security headers, /ws endpoint
backend/api/	Routers: health, live, ml, agents, response, simulation, hqrl, emergency, sources, rescue_desk, voice_agent, dams
backend/services/	Pipeline, weather, hydrology, decision, shelter, route, priority, SOS clustering, person_ops, rescue desk, dam risk, voice agent, algorithm arena, report
backend/ml/	Inference, features (v1–v4), forecast_v4, risk engine, SHAP explanations, change monitor
backend/agents/	Agent graph and LLM agent radio
backend/optimization/	QUBO cost matrix and solvers
backend/simulation/	Digital-twin engine and HQRL benchmark engine
backend/thresholds.py	Single source of truth for all decision thresholds
backend/tests/	pytest suites (API, core, consistency, dams, ML, rescue ops, simulation)
frontend/src/pages/	LiveCommandCenter, LiveResponse, SimulationLab, RescueRoute, HqrlDemo
frontend/src/components/	Maps, charts, agent list, dam monitor, scenario panels, rescue panels
frontend/src/state/	ScenarioContext and rescueOps view model with tests
models/	Model artefacts, metadata and evaluation JSON files
scripts/	Training, benchmarking and VPS deployment scripts
dataset/	INDOFLOODS and CAMELS-IND files (training only)

The following excerpts are taken directly from the repository.

Code 4.1 Gated decision policy, simplified (backend/services/decision_engine.py)

```
if flood_probability < monitor:
    action = "MONITOR"
elif flood_probability < auto:
    action = "HUMAN_REVIEW"
else:
    action = "AUTOMATED_RESPONSE"
    if not (settings.automation_enabled and data_quality_ok
            and model_ok and system_ok):
        action = "HUMAN_REVIEW"
        reason += " - safety gate blocked automation"
```

The policy never acts automatically unless automation is enabled and the data, model and database gates are healthy; the reason string is stored with every decision.

Code 4.2 Flood-aware edge cost (backend/services/route_engine.py)

```
RESCUE_SPEED_KMH = 25.0

def _edge_cost(u, v, road, dest_lat, dest_lon):
    if road and (road.get("blocked")
                 or not road.get("accessible", True)):
        return None          # blocked edge is removed from the graph
    dist = haversine_km(*GRAPH_NODES[u], *GRAPH_NODES[v])
    traffic = float((road or {}).get("traffic") or 0.4)
    flood = float((road or {}).get("flood_exposure") or 0.2)
    return dist * (1.0 + 0.8 * traffic + 1.4 * flood)
```

Code 4.3 QUBO cost matrix (backend/optimization/solvers.py)

```
for i, z in enumerate(zones):
    for j, s in enumerate(shelters):
        dist = haversine_km(z["lat"], z["lon"], s["lat"], s["lon"])
        travel = dist * (1.0 + 0.7 * traffic)
        flood = float(s.get("flood_risk") or 0) * 8.0
        seats = max(int(s.get("capacity", 0))
                    - int(s.get("occupancy", 0)), 1)
        overload = max(0.0, z["population"] / seats)
        c[i, j] = (0.30*travel + 0.25*dist + 0.20*flood
                   + 0.10*traffic*5 + 0.15*overload)
```

Code 4.4 Rescue clock (backend/services/person_ops.py)

```
# queued -> assigned -> en_route -> to_shelter -> rescued
TIME_SCALE = 120.0       # 1 wall second = 2 simulated minutes
DISPATCH_SECONDS = 3.0   # wall s between successive admin dispatches
MOBILISE_SECONDS = 2.0   # wall s crew roll-out once assigned
```

Code 4.5 WebSocket reconnect with back-off (frontend/src/services/ws.js)

```
function scheduleReconnect() {
  if (intentionalClose || refCount <= 0) return;
  clearTimers();
  const delay = Math.min(15000, 800 * 2 ** Math.min(tries, 4));
  tries += 1;
  reconnectTimer = setTimeout(openSocket, delay);
}
```

4.14 User Interface / Application Screens

The user interface has a common top bar with the AquaSentinel brand, a LIVE OPS or SIM MODE badge, the active ML model version (read from the backend), a light/dark theme toggle and navigation. The main screens are summarised in Table 4.7.

Table 4.7 Application screens

Screen (route)	Main content
Live Command Center (/)	Weather, rainfall, river and dam cards with source badges; ML and operational flood probability; agent list and agent radio; flood map; measured latencies
Live Response (/response)	Incidents; policy decision with approve/reject; shelter ranking; flood-aware routes; optimisation results
Simulation Lab (/simulation)	Scenario picker and inputs; RUN SCENARIO; digital-twin outputs; Dam Monitor; ML benchmark; HQRL benchmark and ablation
Rescue Route (/rescue)	Live rescue map with animated units; live SOS roster with stage badges; event feed; voice SOS

Figure 4.6 shows the Live Command Center during a live run. The status bar confirms that the backend and WebSocket are connected, and the KPI cards show live rainfall, the Yamuna stage at ITO, the hybrid flood probability and the risk level. The panel below compares the raw ML probability with the operational risk and prints the exact formula used.

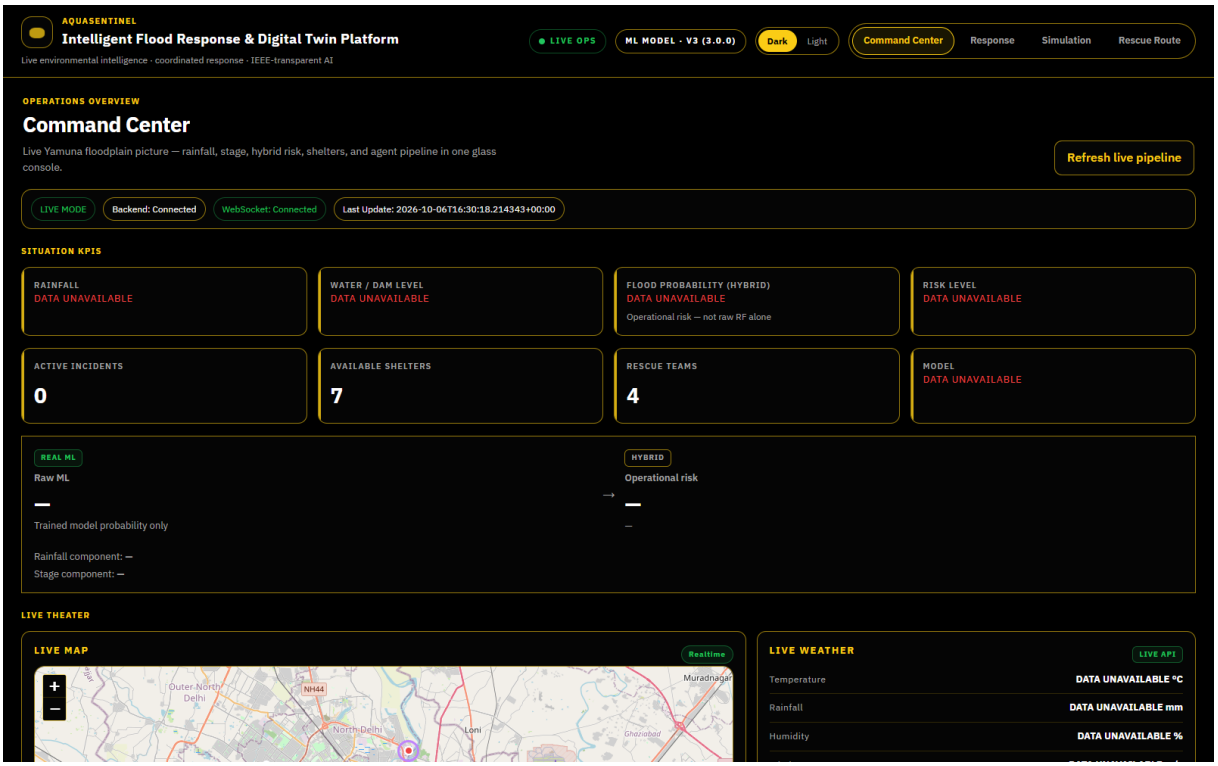


Figure 4.6 Live Command Center screen

Figure 4.7 shows the Simulation Lab after running the Heavy Rainfall scenario. The result cards give the reservoir risk, projected peak level, release and inflow, the best evacuation planner and the safe-evacuation percentage. Below them, the Dam Monitor

shows the calculated reservoir fill for Bhakra Dam beside the ML risk prediction, and the advanced risk engine combines the two.

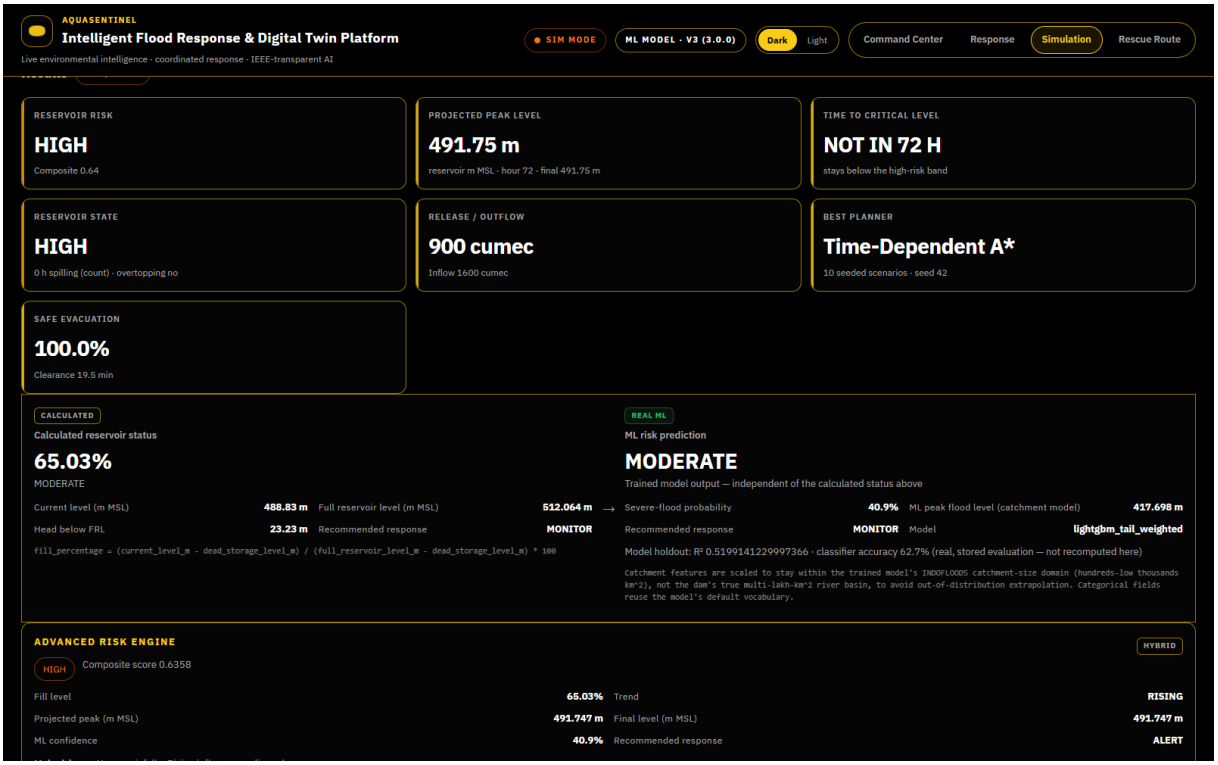


Figure 4.7 Simulation Lab – Heavy Rainfall scenario results and Dam Monitor

Figure 4.8 shows the Rescue Route screen after the simulation residents were pulled into the rescue desk. SOS locations appear as pins on the Delhi map, coloured by status, and the person-case table lists each case with its status, assigned ambulance and shelter.

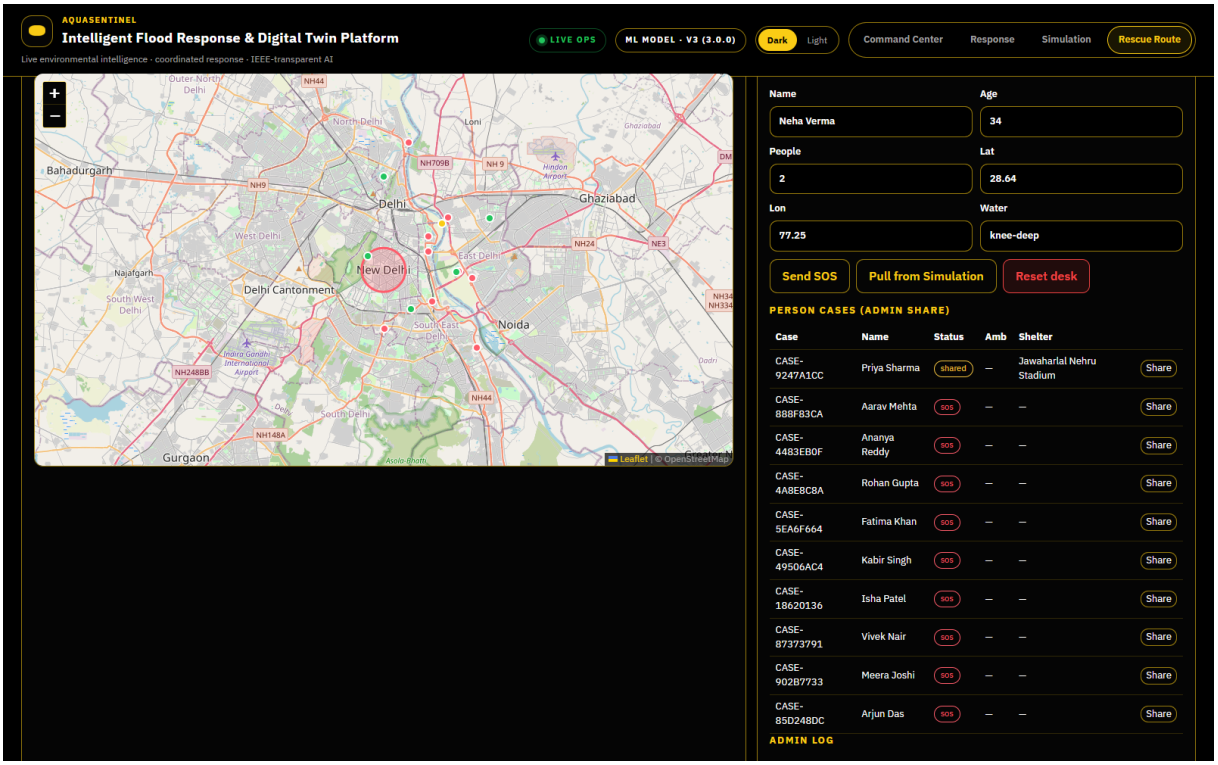


Figure 4.8 Rescue Route – live map and SOS case roster

4.15 Chapter Summary

This chapter presented the design and implementation of AquaSentinel: a five-layer architecture centred on one shared pipeline, ten modules, the algorithms for risk, policy, routing, optimisation, triage and constraint-aware RL routing, the datasets and leakage-free preprocessing, four model versions, the design diagrams, the code organisation and the user interface.

5. CHAPTER 3 – RESULTS AND DISCUSSION

5.1 Introduction

This chapter reports the measured results of the flood models, the 20-model benchmark, the HQRL routing benchmark and the automated tests. All numbers are read from the evaluation files in the models folder or produced by running the repository code with fixed seeds; none are estimated.

5.2 Experimental Setup

Training and benchmarks were run on the development machine described in Table 4.1 (Intel Core i5-1135G7, 8 GB RAM, CPU only) using the Python libraries in Table 4.2. INDOFLOODS models used a held-out set of 910 events (v1) or 31 unseen gauges with 788 events (v2/v3). The v4 model used 1,641,197 training rows (seen gauges, 1980–2008), 349,645 validation rows (2009–2014), a primary test of 52,994 rows from 49 unseen gauges (2015–2020) and a secondary test of 209,149 rows from seen gauges. The 20-model benchmark trained every model on the same 100,000-row sample for fairness. The HQRL benchmark used 30 synthetic scenarios with base seed 42 and the default configuration (6 groups, 3 shelters, flood severity 0.35, traffic 0.40, 8 vehicles).

5.3 Evaluation Metrics

- Accuracy = (TP + TN) / N, and balanced accuracy = (recall + specificity) / 2, which is the main metric because floods are rare (5.5 % of test days).
- Precision = TP / (TP + FP), recall = TP / (TP + FN) and F1-score, their harmonic mean. Recall is critical because a missed flood is costlier than a false alarm.
- ROC-AUC and PR-AUC summarise ranking quality across thresholds; PR-AUC is informative under class imbalance.
- Brier score measures calibration of the probabilities.
- MAE, RMSE and R^2 evaluate the peak-flood-level regressor.

- For routing: safe-evacuation percentage, unsafe-route percentage, mean clearance time, capacity violations and replanning latency.

5.4 Experimental Results

5.4.1 Version 1: INDOFLOODS Severe-Flood Classifiers

Table 5.1 v1 test metrics on 910 held-out events

Model	Accuracy	Precision	Recall	F1	ROC-AUC	Brier
Random Forest (selected)	0.768	0.699	0.620	0.657	0.818	0.163
XGBoost (threshold 0.64)	0.758	0.702	0.564	0.626	0.806	0.182

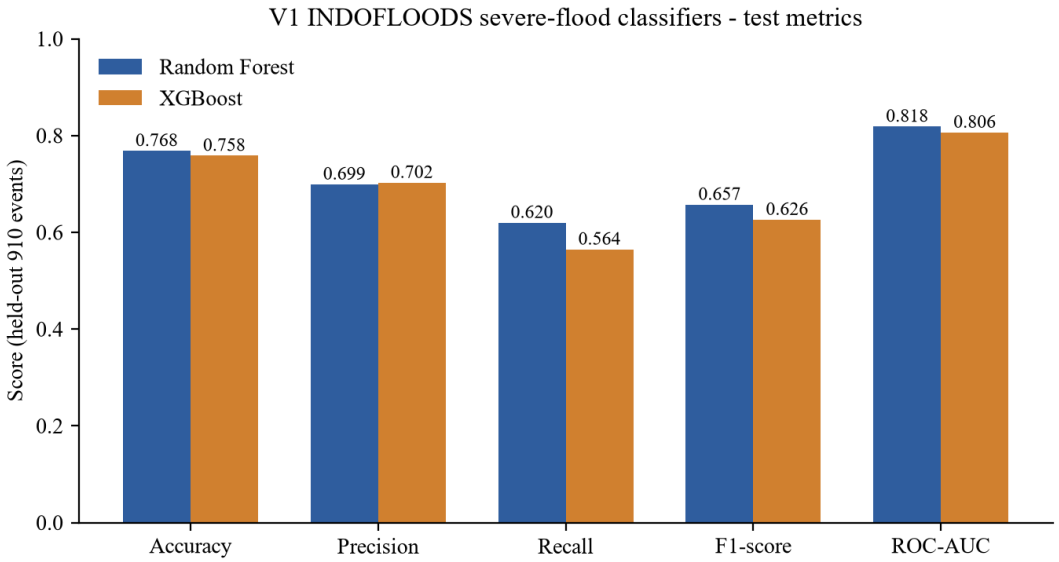


Figure 5.1 v1 classifier test metrics

A gauge-majority classifier already scores about 75.7 % on this label, so the 76.8 % of the Random Forest is close to the ceiling for this target. For this reason the v1 score is used only as a prior in the operational risk engine.

5.4.2 Versions 2 and 3: Peak Level and Severe Flag on Unseen Gauges

Table 5.2 v2/v3 results on 31 unseen gauges (788 events)

Model	MAE	RMSE	R^2
v2 regressor (LightGBM, log target)	80.08	130.47	0.488
v3 regressor (tail-weighted LightGBM)	82.52	126.35	0.520

The v2 CatBoost severe-flood classifier reached accuracy 0.627, precision 0.478, recall

0.671, F1 0.559 and ROC-AUC 0.695 on the unseen gauges. These much lower numbers compared with v1 show the effect of removing gauge identity: once the model cannot memorise the station, the severe-flood label is hard to predict from rainfall alone.

5.4.3 Version 4: 3-Day Flood Forecast on CAMELS-IND

Table 5.3 v4 candidate models on validation (2009–2014, 349,645 rows)

Candidate	Bal. acc.	Recall	Precision	ROC-AUC	PR-AUC
Logistic regression	0.899	0.916	0.391	0.964	0.751
Random forest	0.905	0.924	0.403	0.969	0.781
Hist. gradient boosting	0.909	0.930	0.408	0.971	0.791
XGBoost depth 4	0.909	0.934	0.401	0.971	0.790
XGBoost depth 6	0.909	0.925	0.420	0.971	0.792
XGBoost depth 8	0.909	0.927	0.416	0.971	0.793
XGBoost depth 6, full refit (selected)	0.911	0.936	0.404	0.972	0.796

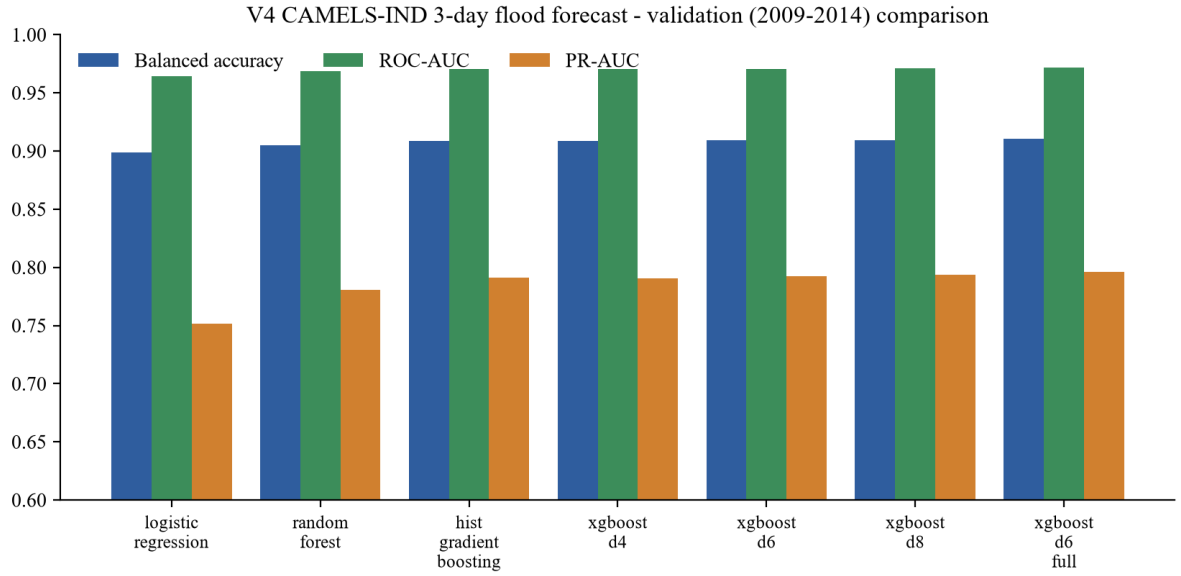


Figure 5.2 v4 candidate comparison on validation data

Table 5.4 v4 test results on 49 unseen gauges (2015–2020, 52,994 rows)

Method	Accuracy	Bal. acc.	Precision	Recall	F1	ROC-AUC
v4 XGBoost (threshold 0.45)	0.894	0.898	0.328	0.902	0.481	0.965
Persistence baseline	0.968	0.765	0.789	0.538	0.640	0.765
Always “no flood”	0.945	0.500	–	0.000	–	–

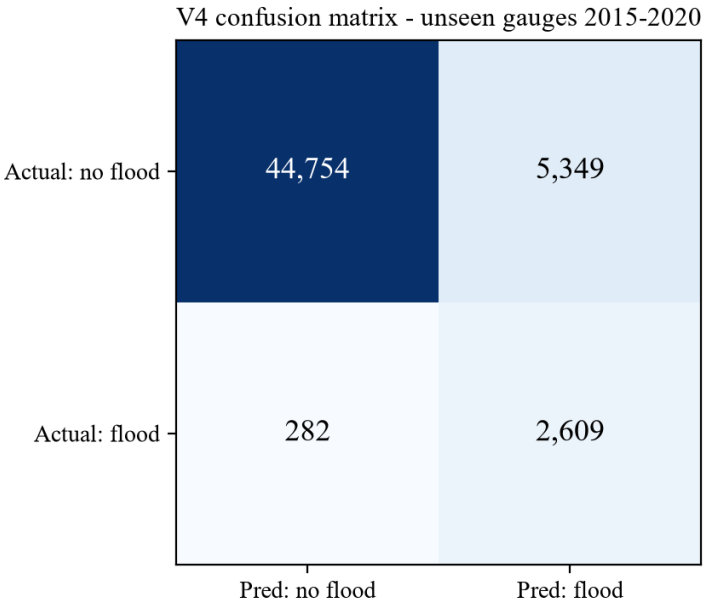


Figure 5.3 v4 confusion matrix on unseen gauges

On gauges never seen in training, the model detected 2,609 of 2,891 flood days (recall 0.902) with a Brier score of 0.065. On flood onsets only (days when the river was not already flooding) recall was 0.813 with ROC-AUC 0.940.

5.4.4 Twenty-Model Benchmark

Twenty classifiers were evaluated with six checks: T1 test accuracy ≥ 0.85, T2 test balanced accuracy ≥ 0.85, T3 train–test balanced-accuracy gap ≤ 0.05, T4 train balanced accuracy ≥ 0.80, T5 beats the baseline and T6 validation–test gap ≤ 0.05 on unseen gauges. On the old INDOFLOODS task no model passed more than three checks; the best test balanced accuracy was 0.647 (RBF SVM), and every model was underfit or overfit. On the CAMELS-IND forecast task, 14 of 20 models passed all six checks, and test balanced accuracy ranged from 0.865 to 0.896 (Random Forest and Bagged Trees 0.896). This confirms that the improvement came from choosing a forecastable target and richer data, not from a particular algorithm.

5.4.5 HQRL Routing Benchmark

Table 5.5 HQRL benchmark, 30 synthetic scenarios, seed 42

Method	Safe evac. (%)	Unsafe route (%)	Clearance (min)	Capacity viol.	Route changes
Static shortest path	90.0	10.0	16.63	0.20	0.00
Time-dependent A*	100.0	0.0	17.81	0.00	0.23
Risk-aware greedy	90.0	10.0	16.63	0.20	0.00
PPO (RL only)	90.0	10.0	16.63	0.20	0.00
PPO + classical QUBO	100.0	0.0	17.81	0.00	0.20
PPO + QAOA (HQRL)	100.0	0.0	17.82	0.00	0.23

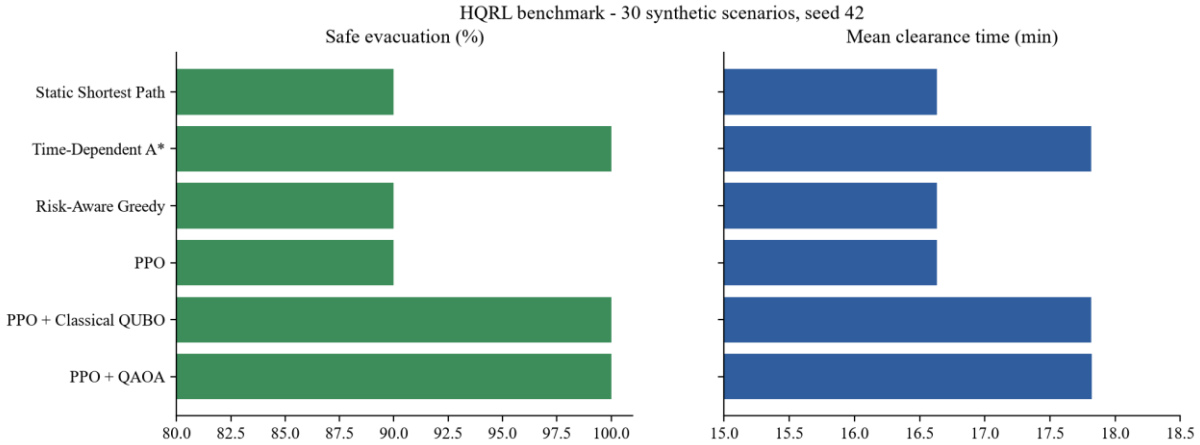


Figure 5.4 HQRL benchmark: safe evacuation and clearance time

In the ablation over 20 scenarios, RL without the safety layer achieved 92.5 % safe evacuation, 7.5 % unsafe routes and 0.15 capacity violations, whereas full HQRL achieved 100 %, 0 % and 0.00 respectively, at the cost of about one extra minute of clearance time (17.57 versus 16.58 min). Replanning latency is a few milliseconds to about 20 ms per decision on the development machine; latency values depend on the hardware and are reported only as indicative.

5.4.6 Automated Tests

The backend pytest suite (API, core logic, data consistency, dam physics, dams, ML inference, v4 ML, rescue operations and simulation) passed 75 of 75 tests. The frontend Vitest suite passed 66 of 68 tests; the two failures are in DamMonitor.test.jsx, where the test's mock Leaflet map does not implement setView. This is a gap in the test mock, not in the application logic, and it is listed as future work.

5.5 Comparison with Existing Techniques

Table 5.6 Comparison with existing approaches

Capability	Gauge thresholds	Stand-alone ML forecast	Static GIS plan	AquaSentinel
Forecast lead time	None (reactive)	Hours to days	None	3 days (v4) + live risk
Validated on unseen gauges	–	Often not	–	Yes (bal. acc. 0.898)
Explanation of prediction	Rule	Rarely	–	TreeSHAP on serving model
Decision with safety gates	Manual	No	Manual	MONITOR / REVIEW / AUTO + gates
Routes react to blockages	No	No	No	Yes, recomputed at pickup
Hard capacity / closure constraints	–	–	Partly	Yes (QUBO + validation)
Digital-twin rehearsal	No	No	No	Same pipeline on scenarios

Against the persistence baseline, the v4 model trades some raw accuracy (0.894 versus 0.968) for a much higher balanced accuracy (0.898 versus 0.765) and recall (0.902 versus 0.538); persistence looks accurate only because most days are dry. Against RL-only routing, the HQRL safety layer removes all unsafe routes and capacity violations in the benchmark.

5.6 Discussion of Results

The results support three conclusions. First, the choice of target matters more than the choice of algorithm: the INDOFLOODS severe label is essentially a station property, while the CAMELS-IND three-day exceedance task is forecastable by many model families. Second, the v4 model favours recall, which is appropriate for warnings, but its precision of 0.33 means that roughly two of three alerts are false alarms; this is why the system routes such probabilities through a human-review band rather than acting automatically at low confidence. Third, hard constraints are what make learned routing safe: in the benchmark the speed advantage of RL-only routing comes entirely from using routes that a validator would reject.

These findings relate directly to the limitations identified in Chapter 3. Fragmentation is resolved by the shared pipeline, optimistic evaluation by unseen-gauge testing, opacity by SHAP on the serving model, silent automation by the gates, and static plans by route recomputation.

Limitations remain. The v4 model is served as a historical replay because no live observed-gauge feed is available; live probabilities therefore rely on the hybrid operational risk engine. The road graph and shelter data are curated rather than live. The HQRL results come from a small synthetic graph, and the QAOA solver is a classical simulation. Dam figures are approximate public reference values, labelled as such in the interface.

5.7 Chapter Summary

The v4 XGBoost forecast reached 0.898 balanced accuracy, 0.902 recall and 0.965 ROC-AUC on unseen gauges; 14 of 20 models passed all six benchmark checks on the CAMELS-IND task; the HQRL safety layer gave 100 % safe evacuation with zero capacity violations; and the automated tests confirm the behaviour of the backend and nearly all of the frontend.

6. CHAPTER 4 – CONCLUSION AND FUTURE SCOPE

6.1 Summary of the Project

The project set out to connect flood forecasting with the decisions that follow it. AquaSentinel implements a single Sense → Predict → Reason → Decide → Optimize → Respond pipeline that serves both live operations and a digital twin. It ingests live weather and river data with explicit source labels, predicts flood risk with models trained and tested without leakage, decides under explicit safety gates, coordinates seven agents through an auditable event graph, computes shelters and flood-aware routes, solves a QUBO assignment with several solvers, triages SOS calls vulnerable-first and tracks every rescue through a stage machine driven by route travel times.

The important outcomes are:

- A production 3-day flood forecast with 0.898 balanced accuracy, 0.902 recall and 0.965 ROC-AUC on unseen gauges.
- A transparent, monotonic operational risk engine and a gated decision policy with audit logging.
- Flood-aware routing and QUBO-based shelter assignment that react to blockages and capacity.
- A constraint-aware HQRL routing pipeline that achieved 100 % safe evacuation in a seed-fixed benchmark.
- A digital twin, a five-dam monitor, a voice SOS agent and a four-screen React dashboard driven entirely by backend state.
- 75 passing backend tests, 66 of 68 passing frontend tests and Docker-based deployment.

Each objective in Section 1.4 is met: live ingestion with provenance (Module 1), leakage-free models with faithful explanations (Module 2), a gated policy (Module 3), agent coordination (Module 4), routing and optimisation (Module 5), SOS triage and rescue tracking (Module 6), and the digital twin with reproducible benchmarks

(Modules 7 and 8).

6.2 Conclusion

AquaSentinel shows that a flood-response platform can be both automated and trustworthy when every value comes from one authoritative state, every model is evaluated on data it has never seen, every learned component is followed by hard constraints, and every automatic action is gated and logged. The project also shows the value of honest reporting: the INDOFLOODS experiments revealed a label that cannot be predicted well from rainfall, which led to the CAMELS-IND forecasting task and a substantially better model.

6.3 Future Scope

- Integrate authenticated India-WRIS and CWC gauge feeds so that the v4 model can run live instead of as a historical replay.
- Replace the curated road graph with OpenStreetMap road data and a live traffic feed, and add road-level inundation from flood maps.
- Extend the forecast to more regions, longer lead times and graph or sequence models that use upstream gauges [33], [24].
- Run the QUBO formulation on real quantum annealing or gate-based hardware and compare it fairly with classical solvers [2].
- Train the evacuation policy on realistic city-scale road networks and population data.
- Add multilingual voice SOS (for example Hindi and Tamil), SMS fallback and offline operation.
- Fix the Leaflet test mock so that the full frontend suite passes, and add end-to-end browser tests.
- Run field exercises with emergency professionals to co-design the interface, as recommended by [30].

SDG GOALS

This project contributes to the United Nations Sustainable Development Goals [32] and to the priorities of the Sendai Framework for Disaster Risk Reduction [31].

SDG 11 – Sustainable Cities and Communities (Target 11.5)

Target 11.5 aims to significantly reduce the number of deaths and people affected by disasters, including water-related disasters. AquaSentinel supports this by forecasting floods three days ahead, choosing safe shelters with free capacity, computing flood-aware evacuation routes that avoid blocked roads, and prioritising children and elderly people in rescue dispatch. Target 11.b, on integrated policies for resilience to disasters, is supported by the gated, auditable decision policy.

SDG 13 – Climate Action (Target 13.1)

Target 13.1 calls for strengthening resilience and adaptive capacity to climate-related hazards. Climate change is increasing the intensity of extreme rainfall. The system turns rainfall, river-stage and reservoir data into early warnings and rehearsable response plans, and its digital twin lets authorities test plans for extreme-rainfall, emergency-release and worst-case-inflow scenarios before they happen.

SDG 3 – Good Health and Well-being (Target 3.d)

Target 3.d aims to strengthen capacity for early warning and management of health risks. The rescue desk assigns ambulances, tracks each person until they reach a shelter, and the voice SOS agent lowers the barrier to asking for help during an emergency.

SDG 9 – Industry, Innovation and Infrastructure (Target 9.1)

Target 9.1 promotes reliable and resilient infrastructure. The Dam Monitor tracks live-storage fill and response levels for five major reservoirs, and the routing engine models how flooding affects the road network, supporting resilient infrastructure planning.

PUBLICATIONS

Nil – no publication has been made from this work at the time of submission.

REFERENCES

[1] Breiman, L. 2001, 'Random forests', Machine Learning, vol. 45, no. 1, pp. 5–32.

[2] Cadeliña, A.R., Cuevas, M.A., Kallos, M., Montemayor, M.J., Uy, R.J., Rodriguez, J.B. & Tubola, O. 2024, 'D-Wave implementation of quantum annealing for optimal resource allocation in disaster response operation of Marikina City', ECTI Transactions on Computer and Information Technology, vol. 18, no. 1, pp. 24–33.

[3] Chen, T. & Guestrin, C. 2016, 'XGBoost: A scalable tree boosting system', Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785–794.

[4] Dijkstra, E.W. 1959, 'A note on two problems in connexion with graphs', Numerische Mathematik, vol. 1, no. 1, pp. 269–271.

[5] Farhi, E., Goldstone, J. & Gutmann, S. 2014, 'A quantum approximate optimization algorithm', arXiv preprint arXiv:1411.4028.

[6] Grieves, M. & Vickers, J. 2017, 'Digital twin: Mitigating unpredictable, undesirable emergent behavior in complex systems', in Transdisciplinary Perspectives on Complex Systems, eds. F.-J. Kahlen, S. Flumerfelt & A. Alves, Springer, Cham, pp. 85–113.

[7] Hagberg, A.A., Schult, D.A. & Swart, P.J. 2008, 'Exploring network structure, dynamics, and function using NetworkX', Proceedings of the 7th Python in Science Conference (SciPy 2008), pp. 11–15.

[8] Hart, P.E., Nilsson, N.J. & Raphael, B. 1968, 'A formal basis for the heuristic determination of minimum cost paths', IEEE Transactions on Systems Science and Cybernetics, vol. 4, no. 2, pp. 100–107.

[9] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q. & Liu, T.-Y. 2017, 'LightGBM: A highly efficient gradient boosting decision tree', Advances in Neural Information Processing Systems 30, pp. 3146–3154.

[10] Kirkpatrick, S., Gelatt, C.D. & Vecchi, M.P. 1983, 'Optimization by simulated annealing', Science, vol. 220, no. 4598, pp. 671–680.

[11] Kirschstein, N. & Sun, Y. 2024, 'The merit of river network topology for neural flood forecasting', Proceedings of the 41st International Conference on Machine Learning (ICML 2024), Vienna.

[12] Kuntla, S.K. & Saharia, M. 2025, 'INDOFLOODS: A comprehensive database for flood events in India enhanced with catchment attributes', Bulletin of the American Meteorological Society. Dataset available from: https://zenodo.org/records/14584655 [Accessed 6 Oct 2026].

[13] Liu, C., Li, Y., Qin, H., Li, W., Mu, L., Wang, S., Liu, D. & Zhou, K. 2025, 'Effective storm surge risk assessment and deep reinforcement learning based evacuation planning: a case study of Daya Bay Petrochemical Industrial Zone', Natural Hazards and Earth System Sciences, vol. 25, pp. 4767–4786.

[14] Liu, F.T., Ting, K.M. & Zhou, Z.-H. 2008, 'Isolation forest', Proceedings of the 8th IEEE International Conference on Data Mining (ICDM 2008), pp. 413–422.

[15] Lucas, A. 2014, ‘Ising formulations of many NP problems’, Frontiers in Physics, vol. 2, article 5.

[16] Lundberg, S.M. & Lee, S.-I. 2017, ‘A unified approach to interpreting model predictions’, Advances in Neural Information Processing Systems 30, pp. 4765–4774.

[17] Lundberg, S.M., Erion, G., Chen, H., DeGrave, A., Prutkin, J.M., Nair, B., Katz, R., Himmelfarb, J., Bansal, N. & Lee, S.-I. 2020, ‘From local explanations to global understanding with explainable AI for trees’, Nature Machine Intelligence, vol. 2, no. 1, pp. 56–67.

[18] MacQueen, J. 1967, ‘Some methods for classification and analysis of multivariate observations’, Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, vol. 1, pp. 281–297.

[19] Mangukiya, N.K., Kumar, K.B., Dey, P., Sharma, S., Bejagam, V., Mujumdar, P.P. & Sharma, A. 2025, ‘CAMELS-IND: hydrometeorological time series and catchment attributes for 228 catchments in Peninsular India’, Earth System Science Data, vol. 17, pp. 461–491.

[20] Morales, J., Kaltenbrunner, A., Lapedriza, À. & Llort, X. 2026, ‘Using machine learning for the prediction of flood-related 112 calls’, Natural Hazards and Earth System Sciences, vol. 26, pp. 3943–3967.

[21] Mosavi, A., Ozturk, P. & Chau, K.-W. 2018, ‘Flood prediction using machine learning models: Literature review’, Water, vol. 10, no. 11, article 1536.

[22] Nearing, G., Cohen, D., Dube, V., Gauch, M., Gilon, O., Harrigan, S., Hassidim, A., Klotz, D., Kratzert, F., Metzger, A. et al. 2024, ‘Global prediction of extreme floods in ungauged watersheds’, Nature, vol. 627, pp. 559–563.

[23] Open-Meteo 2026, Weather Forecast API and Flood API documentation, Available from: https://open-meteo.com/en/docs [Accessed 6 Oct 2026].

[24] Pan, X., Mohammadi, N. & Taylor, J.E. 2024, ‘Short-term streamflow and flood forecasting based on graph convolutional recurrent neural network and residual error learning’, arXiv preprint arXiv:2412.04764.

[25] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O. et al. 2011, ‘Scikit-learn: Machine learning in Python’, Journal of Machine Learning Research, vol. 12, pp. 2825–2830.

[26] Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A.V. & Gulin, A. 2018, ‘CatBoost: unbiased boosting with categorical features’, Advances in Neural Information Processing Systems 31, pp. 6638–6648.

[27] Raei, B., Safarzadeh, R. & Wang, X. 2026, ‘Congestion-aware multi-agent reinforcement learning for wildfire evacuation routing’, International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences, vol. XLIX-M-1-2026, pp. 41–50.

[28] Schulman, J., Wolski, F., Dhariwal, P., Radford, A. & Klimov, O. 2017, ‘Proximal policy optimization algorithms’, arXiv preprint arXiv:1707.06347.

[29] Sultimov, R., Mozikov, M., Abramov, D., Kovalchuk, M., Malykh, M., Makarov, I., Osiptsov, A., Volkov, A. & Maximov, Y. 2026, ‘RESPOND: Realistic environment simulation of population and natural disasters with LLM-driven agents’, Proceedings of the AAAI

Conference on Artificial Intelligence, vol. 40, no. 48, pp. 41694–41696.

[30] Underhill, H., McEwen, L. & Coulthard, T. 2026, '"That's the dream, right?": reflections on the co-design of an environmental digital twin by flood risk management professionals', Frontiers in Environmental Science, vol. 13, article 1724250.

[31] UNDRR 2015, Sendai Framework for Disaster Risk Reduction 2015–2030, United Nations Office for Disaster Risk Reduction, Geneva.

[32] United Nations 2015, Transforming our world: the 2030 Agenda for Sustainable Development, Resolution A/RES/70/1, United Nations General Assembly, New York.

[33] Wang, H., Chen, J., Zheng, Y. & Song, X. 2024, 'Accelerating flood warnings by 10 hours: the power of river network topology in AI-enhanced flood forecasting', arXiv preprint arXiv:2410.05536.

[34] Wang, N., Zhang, H., Dahal, A., Cheng, W., Zhao, M. & Lombardo, L. 2024, 'On the use of explainable AI for susceptibility modeling: Examining the spatial pattern of SHAP values', Geoscience Frontiers, vol. 15, no. 4.

[35] Xie, Y., Jiang, B., Mallick, T., Bergerson, J.D., Hutchison, J.K., Verner, D.R., Branham, J., Alexander, M.R., Ross, R.B., Feng, Y., Levy, L.-A., Su, W. & Taylor, C.J. 2025, 'A RAG-based multi-agent LLM system for natural hazard resilience and adaptation', arXiv preprint arXiv:2504.17200.

[36] Xu, F., Ma, J., Li, N. & Cheng, J.C.P. 2025, 'Large language model applications in disaster management: An interdisciplinary review', International Journal of Disaster Risk Reduction, vol. 127, article 105642.

[37] Yen, J.Y. 1971, 'Finding the k shortest loopless paths in a network', Management Science, vol. 17, no. 11, pp. 712–716.